



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Course Code: CSE 402.

Course Title: Operating System Lab.

Experiment No: 1

Experiment Title: Implementation of First Come First Serve (FCFS) CPU Scheduling Algorithm.

Date of Submission : 04-08-2026

Submitted By:

Name : Shaumi Tabassum Tanha

Student Id : 23101145

Semester : 4th Year 1st Semester

Section : C-2

Submitted To

Aita Rahman Orthi

Lecturer , UAP

Problem Statement:

Implement FCFS CPU Scheduling for the following scenario where No of processes = 5 and Process informations are-

P id	AT	BT
P0	3	1
P1	5	3
P2	2	2
P3	1	2
P4	6	3

Calculate CT, TAT, WT, AVG TAT and AVG WT. Also represent the process execution sequence.

Source Code:

```
processes = ["P3", "P2", "P0", "P1", "P4"]
arrival_time = [1, 2, 3, 5, 6]
burst_time = [2, 2, 1, 3, 3]

n = len(processes)

completion_time = [0] * n
turnaround_time = [0] * n
waiting_time = [0] * n

current_time = 0
```

```

for i in range(n):
    if current_time < arrival_time[i]:
        current_time = arrival_time[i]

    completion_time[i] = current_time + burst_time[i]
    current_time = completion_time[i]

    turnaround_time[i] = completion_time[i] - arrival_time[i]
    waiting_time[i] = turnaround_time[i] - burst_time[i]

print(f"{'Process':<10}{'AT':<8}{'BT':<8}{'CT':<8}{'TAT':<8}{'WT':<8}")
print("-" * 50)

for i in range(n):
    print(f"processes[i]:<10}{arrival_time[i]:<8}{burst_time[i]:<8}"
        f"completion_time[i]:<8}{turnaround_time[i]:<8}{waiting_time[i]:<8}")

avg_wt = sum(waiting_time) / n
avg_tat = sum(turnaround_time) / n

print("-" * 50)
print(f"Average Waiting Time      = {avg_wt:.2f}")
print(f"Average Turnaround Time    = {avg_tat:.2f}")

```

Output:

Process	AT	BT	CT	TAT	WT
P3	1	2	3	2	0
P2	2	2	5	3	1
P0	3	1	6	3	2
P1	5	3	9	4	1
P4	6	3	12	6	3

Average Waiting Time = 1.40					
Average Turnaround Time = 3.60					

Discussion:

In this experiment, the First-Come-First-Served (FCFS) CPU scheduling algorithm was implemented using Python. The process information, consisting of Process ID, Arrival Time (AT), and Burst Time (BT), was initially stored in a list. Before scheduling began, the processes were sorted in ascending order of their arrival times to ensure correct FCFS execution order.

During the simulation, the program continuously monitored the current CPU time. If the CPU became free before the next process had arrived, it remained idle until that process reached the ready queue. Once a process started execution, it ran to completion without interruption (non-preemptive). After each process finished, its Completion Time (CT), Turnaround Time ($TAT = CT - AT$), and Waiting Time ($WT = TAT - BT$) were calculated using the standard FCFS formulas. Finally, the average Turnaround Time and average Waiting Time were computed to evaluate the overall performance of the algorithm.

The results show that all processes were executed strictly according to their arrival order, confirming the non-preemptive nature of FCFS. This experiment demonstrates that FCFS is one of the simplest CPU scheduling algorithms and is easy to implement. However, it suffers from a major drawback known as the **Convoy Effect**. When a process with a long burst time arrives early, subsequent shorter processes are forced to wait, which significantly increases the average waiting time and reduces system responsiveness. This limitation makes FCFS less suitable for interactive or time-sharing systems where fairness and low response time are important.

A simple highlight that in a class room I faced some issue in for loop portion i have modified that in a report attached below

Before

```
for i in range(n):
    if current_time < arrival_time[i]:
        current_time = arrival_time[i]

    completion_time[i] = current_time
    + cuurent_time = completion_time [i]
    turnaround_time = completion_time
    waiting_time = turnaround_time

    print("Process\tAT\tBT\tCT\tAT\tWT")
```

After

```
for i in range(n):
    if current_time < arrival_time[i]:
        current_time = arrival_time[i]

    completion_time[i] = current_time + burst_time[i]
    current_time = completion_time[i]

    turnaround_time[i] = completion_time[i] - arrival_time[i]
    waiting_time[i] = turnaround_time[i] - burst_time[i]

print(f"{'Process':<10}{'AT':<8}{'BT':<8}{'CT':<8}{'TAT':<8}{'WT':<8}')
```

Conclusion:

The objective of this experiment was successfully achieved by implementing the First-Come-First-Served (FCFS) CPU scheduling algorithm in Python. The program correctly sorted the processes according to their arrival times and accurately calculated the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process. It also computed the average Turnaround Time and average Waiting Time, which were found to be 3.60 and 1.40 respectively.

Through this experiment, the fundamental concept of CPU scheduling and the working principle of the FCFS algorithm were clearly understood. The results confirm that FCFS is a simple and fair scheduling technique that executes processes strictly in the order of their arrival without preemption. Due to its straightforward implementation, FCFS is suitable for basic scheduling environments. However, it is not always the most efficient algorithm. When a long process arrives early, it can cause subsequent shorter processes to wait for a longer time—a phenomenon known as the Convoy Effect—leading to higher average waiting time and reduced system responsiveness.

Therefore, while FCFS is easy to understand and implement, more advanced scheduling algorithms such as Shortest Job First (SJF) or Round Robin (RR) are generally preferred in practical operating systems for better performance in terms of reduced waiting time and improved CPU utilization.